MERCHANDISING AI

A Multi-Agent System for Non-Purchaser Feedback Analytics

Technical Report

Architecture · Topology · Agents · Tools · Evaluation · Observability

Prepared by

Sundar Ram Subramanian

Date:

May 16, 2026

Table of Contents

Table of Figures

Table of Tables

Table of Equations

1. Executive Summary

The Merchandising AI is a production-grade multi-agent chat assistant that turns natural-language questions about non-purchase feedback for a six-store jewelry retail chain (stores X1 through X6) into grounded, source-cited answers backed by SQL queries against a MySQL data store. The system is built around a rule-based Python Orchestrator that coordinates nine specialized Large Language Model (LLM) agents: **five reasoning agents** and **four reflection (review) agents,** together with **eleven deterministic tools** that perform **parsing**, **safety enforcement, SQL execution, sandboxed visualization, Excel rendering, groundedness verification, and persistent trace logging**.

Two model tiers are used: **Anthropic Claude Sonnet 4.6** is used for the Coder and Code Reviewer (the SQL-quality bottleneck) and **Claude Haiku 4.5** is used for every other agent (routing, semantic review, prose composition, visualization sub-pipeline, recovery decisions). All agents run at temperature 0 to maximize determinism, and the SQL pipeline implements a chain-of-thought "reasoning" block whose semantics are independently verified by the Code Reviewer.

Quality is enforced through three concentric **runtime validation rings**: deterministic regex/Python guards, cross-reviewing LLM agents, and bounded retry loops with a Supervisor escape hatch, together with **four out-of-band evaluation suites** that run on a cadence (a 7-check integration smoke test, a 26-entry golden Q-to-SQL regression suite, a 15-prompt manual answer-correctness suite, and an offline topic-classifier accuracy harness). A regex-based groundedness check verifies every numeric token in the final written answer against a candidate set derived from the SQL result rows; if the budget is exhausted, the system hard-blocks the answer rather than ship un-verified prose.

Every chat turn persisted to a dedicated MySQL audit table named chat_trace with derived columns (path, retry count, supervisor invocation, groundedness warning) plus the full JSON step trace. This long-running observability stream is the foundation on which future regression analysis and quality dashboards can be built. The remainder of this report describes the architecture, topology, frameworks, agentic workflow, tool catalogue, reflection design, evaluation strategy, observability, and ethical considerations with the reasoning behind each design choice presented in line with the choice itself.

2. Project Overview and Business Context

The Merchandising AI supports the merchandising and inventory-planning team of a regional jewelry retail chain operating six physical stores labelled X1 through X6. The team's

primary operational challenge is to understand and act on customer non-purchase feedback verbose free-text comments captured at the storefront by the salesman when a visiting customer leaves without buying. Approximately 1,200 such feedback records (synthetic for the present build, with future provisions for production data flow) form the analytical substrate of the system.

The system exposes **two surfaces** to the business user. The **Chat surface** is a free-form question-and-answer interface driven by the **nine-agent pipeline**; this is the most heavily used surface and is the focus of the bulk of this report. The **Recommendations surface** is a **one-click deterministic report** (top focus areas, per-store breakdown, distribution charts) that is computed directly against MySQL using pandas for stakeholder-facing consistency. Showing the same data with the same filters always produces the same report, an explicit non-negotiable for the business audience that views the Recommendations panel.

2.1 Why a Multi-Agent System Was the Right Tool

The business question "what should we stock more of at store X1?" is deceptively complex. It requires translating a colloquial business phrase into a precise **multi-dimensional SQL aggregation, executing that SQL** safely against production data, **semantically verifying** that the rows actually answer the question, **composing prose** that cites only verifiable numbers, optionally **rendering a chart**, and finally **surfacing the entire chain of reasoning** so the user can audit the answer. No single LLM call and no plain retrieval-augmented prompt can satisfy all six obligations simultaneously. Splitting the work across **specialized agents whose outputs are independently reviewed** is the smallest design that achieves all six.

2.2 Use-Case Coverage

Table 1 summarizes the nine canonical use cases that drove the architecture. Each row identifies the path the Planner picks and the agents that ultimately serve the answer.

Table 1 Use-Case Coverage Matrix

#	Use Case	Path	Components Involved
A	Top issue at X1?	sql	Planner → Coder → sql_executor → Writer
B	Follow-up: now show me X4	sql	Planner resolves reference; rest of SQL path

#	Use Case	Path	Components Involved
C	Tell me about my stores	clarify	Planner emits clarifying question
D	Hi / What does X mean?	clarify	Planner emits clarifying question
E	Compare size issues across X1, X4	sql + viz	Full SQL path + Viz sub-pipeline
F	Broken SQL written by Coder	sql + retry	Code Reviewer rejects; Coder retries (≤5)
G	SQL ran but answered wrong question	sql + retry	Output Reviewer rejects; Coder retries (≤2)
H	Show Recommendations	deterministic	Bypasses agents; pandas pivot tables
I	Pre-flight topic enrichment	offline	data_prep/02_enrich_topics.py (one-time)

3. High-Level System Architecture

The system is best understood as a tightly stratified set of eight layers: **Data, Knowledge, Memory, Reasoning, Reflection, Orchestration & Recovery, Tooling & Validation, and Observability & Interface**. Each layer has a single concern, and each layer talks only to its neighbors via a narrow interface. The Orchestrator sits at the heart of the system and is the only component permitted to call agents or tools; this **single-hub discipline** is what makes the system auditable, retry-budgeted, and bounded in worst-case cost.

3.1 Foundational Design Choices

Five cross-cutting choices touch every layer and deserve attention up front before details obscure them.

- **Determinism over creativity.** All nine LLM agents run at temperature 0.0. The Writer is the most consequential beneficiary of this rule because of where it sits in the pipeline: everything upstream of it, the SQL rows from sql_executor, the safe-derivations list computed in plain Python are already deterministic, so the Writer is the single remaining

source of probabilistic variance before the answer reaches the user. The Writer was originally configured at temperature 0.2 for natural prose, but at that setting the same SQL result of 41 could yield "approximately 40," "exactly 41," or a fabricated subset sum on different retries even though the underlying number was fixed. Dropping to 0.0 pins the Writer's sampling and removes most of that variance; the groundedness check (Section 15.4) handles whatever residual non-determinism survives.

- **Rule-based Orchestrator, not an autonomous-agent loop.** The control plane is pure Python with no LLM in the routing logic. Agents emit structured text wrapped in XML-style tags or JSON verdicts; the Orchestrator parses and routes the next step deterministically. The system deliberately rejects the Model Context Protocol (MCP) / tool-use pattern in favor of this single-hub topology because it makes worst-case cost bounded, retry budgets reliable, and debugging tractable.
- **Closed enums everywhere.** The Topic Classifier, every reviewer's verdict, the Planner's path, and the Supervisor's action are all enum-checked. Out-of-enum output is rejected as a transient failure (counted against the relevant retry budget). Without closed enums, the system would have to do natural-language understanding on every LLM output to decide "did the reviewer say ok or not?" That's both expensive (another LLM call) and unreliable.
- **Hard-block over soft-block for numeric grounding.** When the Writer produces a number that does not reconcile against the result rows after two attempts, the system ships an explicit "I couldn't produce verifiable numbers" message rather than the unverified prose.
- **Schema vocabulary discipline.** The column originally named attribute_type was renamed non_purchase_type because the everyday word "attributes" naturally collided with both columns and caused the LLM to disambiguate incorrectly. Schema-level naming clarity beats prompt-level scaffolding.

3.2 Why a Single-Hub Orchestrator over Tool-Use / MCP

The most consequential architectural choice in the system is the rejection of the autonomous agent loop. Modern frameworks (LangGraph, AutoGen, the Anthropic Tool Use API, and Model Context Protocol servers) let a single LLM decide at each step which tool to invoke next. That style buys flexibility: the agent can navigate problems it has not been programmed for, but at the cost of predictability, retry-budget enforcement, cost ceilings, and debuggability. For a focused merchandising chat where the same control flow applies to every turn, autonomy is the wrong trade-off.

The chosen pattern instead has the LLMs emit structured text. For example, a <sql_query>...</sql_query> block, or a JSON verdict object and the Orchestrator's Python code

parses and dispatches the next step. The Coder has no awareness that sql_safety_guard or sql_executor exist. This separation has five concrete benefits: every retry budget is enforced by Python; every tool call goes through a single auditable control-flow point; tracing and trace persistence attach cleanly; no tool-use round trips inflate cost; and debugging is a matter of reading actual Python files instead of decoding an LLM's tool-use rationale.

3.3 The Eight-Layer Stack

The eight conceptual layers map to specific source modules as follows:

- **Data Layer:** MySQL non_purchasers_feedback and chat_trace tables.
- **Knowledge Layer:** Offline topic-enrichment pipeline data_prep/02_enrich_topics.py and the SCHEMA_TEXT constant in tools/sql_tools.py.
- **Memory Layer:** Streamlit session state, the Planner's eight-message look-back, and the resolved_question mechanism that bakes context once.
- **Reasoning Layer:** Planner, Coder, Writer, Viz Coder, Supervisor.
- **Reflection Layer:** Code Reviewer, Output Reviewer, Viz Code Reviewer, Viz Reviewer.
- **Orchestration & Recovery:** Orchestrator allowing retry budgets, the Supervisor escape hatch, and the hard-block groundedness loop.
- **Tool & Validation:** sql_parser, sql_safety_guard, sql_executor, viz_code_parser, viz_generator, to_excel, groundedness_check, safe_derivations_summary, log_turn, ensure_table, get_schema_for_prompt.
- **Observability & Interface:** StepEvent dataclass, the chat_trace table, the Streamlit chat / Recommendations UI.

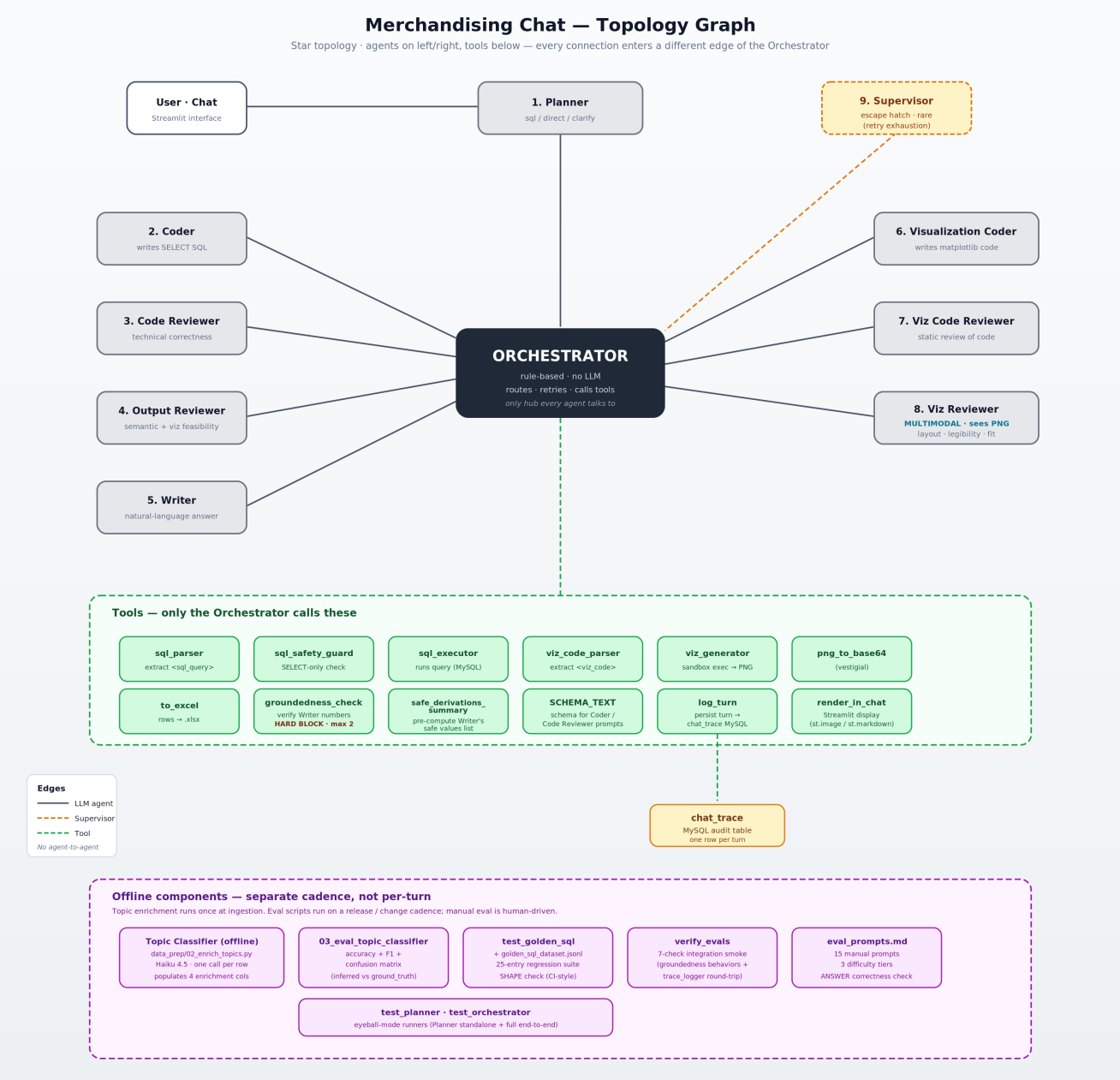
4. Topology and Component Graph

Figure 1 renders the topology of the system with the Orchestrator at the hub. Every agent is connected only to the Orchestrator (never to another agent), and every deterministic tool is connected only to the Orchestrator (never invoked directly by an agent). This visually confirms the single-hub design discussed in Section 3.

The five reasoning agents (Planner, Coder, Writer, Viz Coder, Supervisor) and the four reflection agents (Code Reviewer, Output Reviewer, Viz Code Reviewer, Viz Reviewer) occupy the eastern and western sides of the orchestrator. The deterministic tools fan out below. Every connection is to the central Orchestrator, illustrating that no agent shortcuts the orchestrator.

This pattern is consistent with the chosen single-hub control plane and contrasts deliberately with peer-to-peer agent topologies common in autonomous agent frameworks.

Figure 1 Component Call Graph showing the Orchestrator as the single hub for nine agents and eleven tools



Visualizations of agent systems frequently show many edges among many nodes: peer-to-peer agent meshes, plan-and-execute loops, or graph-shaped LangGraph state machines. The deliberate "**hub-and-spoke**" shape of Figure 1 is itself the design statement. This topology gives the developer a single place to set retry budgets, a single place to log every step, a single place to apply safety guards, and a single file where the entire control flow can be read top-to-bottom.

5. Technology Stack and Framework Rationale

Table 2 below summarizes the runtime dependencies declared in requirements.txt with the role each library plays. The most consequential choices: model provider, LLM client library, data store, and UI framework are explained after the table.

Table 2 Library Dependencies and Their Roles

Library	Pinned Version	Role in the System
streamlit	>=1.32	User-facing web UI; chat surface plus deterministic Recommendations report
mysql-connector-python	>=9.0	Direct driver to MySQL for sql_executor, trace_logger, enrichment script
pandas	>=2.0	DataFrame I/O for the Excel export and the deterministic Recommendations report
python-dotenv	>=1.0	Loads MYSQL_* and ANTHROPIC_API_KEY at import time from project-root .env
aisuite[anthropic]	>=0.1.7	Provider-agnostic chat completions client; one swap point for future model changes
matplotlib	>=3.7	Backend for the sandboxed viz_generator that renders PNG charts
numpy	>=1.24	Pre-loaded into the viz sandbox as np; available to Viz Coder code
openpyxl	>=3.1	Workbook writer behind to_excel; produces .xlsx download bytes

5.1 Model Provider: Anthropic Claude

Anthropic Claude is the LLM provider for the entire system. Within Claude, the system uses a deliberate **two-tier strategy**: **Claude Sonnet 4.6** - the smarter and more expensive tier is used to solve the SQL bottleneck, and **Claude Haiku 4.5** - roughly three to five times cheaper is used everywhere else, where it delivers Sonnet-acceptable quality on simpler tasks. A vision-

capable variant of Haiku 4.5 is also used for the one job that needs to inspect an image (the Viz Reviewer). The table below maps every agent in the system to the specific model that runs it.

Table 3 Model-to-Agent Allocation across the System

Agent	Model used	Why this model
Coder	Claude Sonnet 4.6	Writes the SQL; the hardest, highest-stakes task in the system. **Haiku failed here.**
Code Reviewer	Claude Sonnet 4.6	Reviews the Coder's SQL; must match the Coder's tier or it will rubber-stamp subtle errors.
Planner	Claude Haiku 4.5	Routes the turn into sql / direct / clarify; a simple classification task.
Output Reviewer	Claude Haiku 4.5	Yes/no judgment on whether the rows answer the question — binary verdict.
Writer	Claude Haiku 4.5	Composes prose from a pre-computed safe-values list; arithmetic is off the LLM.
Viz Coder	Claude Haiku 4.5	Writes short matplotlib snippets — pattern-based, well-bounded.
Viz Code Reviewer	Claude Haiku 4.5	Static safety check for banned identifiers and missing labels.
Viz Reviewer	Claude Haiku 4.5 (vision)	Must SEE the rendered chart PNG — requires multimodal capability.
Supervisor	Claude Haiku 4.5	Picks one of four recovery actions from the step trace; a small decision.
Topic Classifier (offline)	Claude Haiku 4.5	One-time classification into a 10-topic closed enum; cheap and reliable.

Why Sonnet 4.6 for the Coder and Code Reviewer. The Coder is the system's bottleneck. Translating a compound business question such as “compare size issues between X1 and X4 broken down by product” into a correct multi-line MySQL query with appropriate joins, group-bys, window functions and the right column choice is the hardest single task in the pipeline.

Earlier iterations of the system ran the Coder on Haiku 4.5 to save cost, but Haiku failed repeatedly on two specific things: window-function syntax (constructs such as ROW_NUMBER() OVER (PARTITION BY … ORDER BY …) needed for top-N within each group queries) and the non_purchase_type versus attribute_value disambiguation that is central to the domain vocabulary. Sonnet 4.6 handles both reliably under the same prompt. The Code Reviewer is paired with the Coder at the same tier because a reviewer weaker than the agent it reviews cannot catch that agent's subtle mistakes. Tier-matching the reviewer is what makes the reasoning-versus-SQL verification work in practice.

Why Haiku 4.5 for almost everyone else. The remaining agents perform structurally simpler tasks: the Planner classifies the turn into sql / direct / clarify; the Output Reviewer issues a binary verdict on whether the rows answer the question; the Writer composes prose from a pre-computed safe-values list (arithmetic is taken off the LLM and done in plain Python); the Viz Coder writes short pattern-based matplotlib snippets; the Viz Code Reviewer enforces a static safety check; the Supervisor picks one of four recovery actions from a trace. On all of these Haiku 4.5 was tested side-by-side with Sonnet and showed no measurable quality drop. Since Haiku is roughly three to five times cheaper per call, running it on these agents keeps the worst-case turn cost bounded (around 0.15 USD with full retries) instead of multiplying every retry by the Sonnet rate.

Why Haiku 4.5 vision specifically for the Viz Reviewer. The Viz Reviewer's job is to look at the rendered chart PNG and judge layout, label readability, color coherence, and fit to the question. None of this can be done from the matplotlib code alone as code can be perfectly valid Python and still produce a confusing or mislabelled image. The Viz Reviewer therefore needs to accept both text (the resolved question and instructions) and an image (the rendered PNG) in the same call. Claude Haiku 4.5 supports this multimodal input natively, which is what enables the Viz Reviewer to exist as an agent at all. Without it, the system would be limited to static code review only, and a class of code-valid-but-visually-broken charts would slip through to the user.

In summary, the system uses Sonnet 4.6 only where it is structurally required to address the bottle neck (the Coder and the Code Reviewer that polices it), Haiku 4.5 for every other text-only agent (Planner, Output Reviewer, Writer, Viz Coder, Viz Code Reviewer, Supervisor, and the offline Topic Classifier) where it is sufficient and three to five times cheaper, and the Haiku 4.5 vision variant for the one agent that must inspect an image (the Viz Reviewer). The full per-agent hyper-parameters including temperature, max_tokens, prompt-caching strategy are listed in Table 7 in Section 8.

5.2 LLM Client: aisuite with a Native-SDK Escape Hatch

Every agent in the system needs to make HTTP calls to an LLM provider, and how those calls are mediated is a deliberate choice with consequences for portability and vendor lock-in. All LLM-facing code is centralised in a single module, **agents/llm.py**, which exposes a small set of helpers (**call_llm**, **call_llm_json**, **call_llm_vision**, **call_llm_vision_json**, **load_prompt**) that the nine agents import. No agent imports a vendor SDK directly. This single chokepoint is what makes the choices below enforceable.

Every agent except the multimodal Viz Reviewer flow through **aisuite**, a thin provider-agnostic facade that exposes a uniform chat.completions.create(...) interface and dispatches internally to the relevant vendor SDK (Anthropic's anthropic, OpenAI's openai, Google's google-generativeai) based on the model identifier passed in. The model strings themselves live in two module-level constants: MODEL_HAIKU = "anthropic:claude-haiku-4-5-20251001" and MODEL_SONNET = "anthropic:claude-sonnet-4-6", which together form the entire model registry of the system. A future migration to a different provider therefore reduces to editing those two constants, swapping the API key environment variable, and confirming the new models' instruction following quality matches what the prompts assume; the agents, the orchestrator, the tools, and the tests need no changes. Without this layer of abstraction, every agent file would carry a direct dependency on the anthropic SDK and a vendor migration would become a multi-file refactor.

There is a escape hatch that captures the design intent precisely. One deliberate exception, and it is the only place a vendor SDK is imported directly: the **Viz Reviewer's multimodal calls**. The Viz Reviewer must send both a text prompt and a base64-encoded PNG in the same request and receive a structured JSON verdict back. Aisuite's support for Anthropic's multimodal content-blocks format was incomplete. Rather than patch aisuite or build a parallel encoding path that defeated the purpose of the wrapper, the cleaner choice was to bypass aisuite for this single case and call the native anthropic SDK directly. The helpers **call_llm_vision** and **call_llm_vision_json** in agents/llm.py do exactly this: import anthropic, base64-encode the PNG inline, construct the [{"type": "image", ...}, {"type": "text", ...}] content list the native SDK expects, and parse the response. The bypass here is constrained in three ways: it is **isolated by file** (only agents/llm.py imports anthropic; every other file imports from agents.llm), **isolated by function** (only the two vision helpers use the native SDK; the four text helpers still route through aisuite), and **annotated inline** so a future maintainer reading the code immediately understands the bypass is intentional rather than a forgotten artifact. If a later version of aisuite gains complete multimodal support for Anthropic, the bypass collapses back into a uniform path and because

every agent imports only from agents.llm, the rest of the codebase is entirely unaffected by that cleanup.

5.3 Data Store: MySQL with InnoDB

The data substrate is MySQL with the InnoDB engine and utf8mb4 character set. InnoDB is chosen for transactional row-level inserts in the enrichment script (crash-safety per row), and utf8mb4 is chosen because customer feedback can contain emoji and non-Latin characters. The chat_trace audit table sits in the same database as non_purchasers_feedback, which simplifies operational topology with a single connection string, a single backup target, and a single set of read-only credentials for an analyst querying both the source data and the audit history.

5.4 UI Framework: Streamlit

Streamlit was chosen for the user-facing surface because it gave the fastest path from working Python to deployable web UI for a small internal user base. Trade-offs are accepted: full-page reload semantics on widget state changes, limited per-component styling, and a deliberate dependence on st.session_state for short-term chat memory. For a larger user base the project would migrate to a Next.js / FastAPI split, but for the present audience Streamlit's full-stack-in-one-file shape gave the most leverage per hour of engineering.

5.5 The Deliberate Non-Choice: No Vector Database, No RAG

The system does not use a vector database, embedding model, or retrieval-augmented generation. This is a deliberate non-choice. The data is small (about 1,200 rows of structured feedback), the schema is fixed, and the questions are quantitative aggregations rather than semantic search. The right primitive for these conditions is SQL with a strong typed schema, not embeddings and a similarity index. The system would absorb a vector store if the use case widened to include free-text Q&A over arbitrary policy documents or PDFs.

5.6 The Deliberate Non-Choice: No LangChain / No LangGraph

Neither LangChain nor LangGraph are present as dependencies. The agents/orchestrator.py module is approximately 679 lines of plain Python and replaces what would otherwise be a more complex graph-state-machine integration. The trade-off accepted is that adding a new agent requires editing the orchestrator file rather than declaring a new node in a graph. The benefit gained is total visibility into control flow including every retry, every

escape hatch, every step event emission is visible inline in one file and zero exposure to framework breakage when an upstream library tweaks its public interface.

6. Data Layer and Knowledge Layer

The data layer is a synthesized single MySQL table named non_purchasers_feedback containing approximately 1,200 rows. The table carries thirteen columns: nine captured directly at the storefront and four enriched offline by the LLM topic-classifier pipeline (data_prep/02_enrich_topics.py). The enriched columns are what makes the chat agent's SQL clean: instead of fighting LIKE '%design%' patterns, the Coder writes aggregations against structured topic and non_purchase_type columns.

6.1 Schema Inventory

Table 4 lists every column with its source, type, and role.

Table 4 Database Column Inventory

Column	Source	Type	Purpose
feedback_id	Captured	INT PK	Primary identifier
visit_date	Captured	DATE	Day the customer visited the storefront
store_code	Captured	VARCHAR(8)	One of X1..X6; indexed for filter performance
customer_name	Captured	VARCHAR(120)	Customer name as recorded by salesman
customer_email	Captured	VARCHAR(160)	Customer email (PII; see Section 17)
user_category	Captured	VARCHAR(40)	Customer demographic bucket
product_looking_for	Captured	VARCHAR(40)	Indexed; one of five jewelry product types
reason_for_non_purchase	Captured	TEXT	Free-text verbose reason
ground_truth_topic	Captured (synthetic)	VARCHAR(40)	Eval anchor; NOT used in production queries

Column	Source	Type	Purpose
inferred_topic	Enriched	VARCHAR(40)	LLM-classified into 10 canonical topics
topic_confidence	Enriched	FLOAT	Float in [0.0, 1.0]
non_purchase_type	Enriched	VARCHAR(40)	One of 9 categories of non-purchase reason
attribute_value	Enriched	VARCHAR(200)	Normalised phrase, e.g. 'star design' or 'size 8'

6.2 The Closed-Enum Discipline

Both inferred_topic and non_purchase_type are constrained to a closed enum that is enforced in two places: the prompt for the offline Topic Classifier, and a Python validate() function in 02_enrich_topics.py. The dual enforcement is a belt-and-braces protection against LLM vocabulary drift — a freshly-released model might propose plausible-but-novel topic labels that would silently fragment downstream aggregations.

Table 5 Closed Enum — Inferred Topics with their mapped non_purchase_type

Inferred Topic	Mapped non_purchase_type
Design Unavailable	design
Size Unavailable	size
Stock Unavailable	stock
Price Too High	price
Quality Concerns	none
Weight Concerns	weight
Color/Finish Mismatch	color
Customization Not Offered	customization
Sales Service	service
Others	none

Table 6 Closed Enum — Non-Purchase Types

non_purchase_type	Description / Typical attribute_value
design	Specific design pattern (e.g. 'star design', 'jhumka')
size	Specific size with units (e.g. 'size 8', '10 inch')
color	Metal/finish phrase (e.g. 'rose gold', 'white gold')
weight	Weight preference (e.g. 'under 8g', 'lightweight')
price	Budget phrase (e.g. 'under 25k', 'below 1 lakh')
customization	Customization requested (e.g. 'engraving')
service	Service-related issue; attribute_value is NULL
stock	Item simply out of stock; attribute_value is NULL
none	No specific attribute extractable; attribute_value is NULL

6.3 Why Offline Enrichment Rather Than On-Demand Classification

The choice to enrich topics once, at data ingestion, rather than at chat-time was an optimization decision. If enrichment happened inside each chat turn, the Coder would have to write SQL that pattern-matches free-text reason_for_non_purchase columns which would be fragile, slow, and statistically incorrect (any single LIKE pattern misses paraphrased reasons). With enrichment performed once and persisted, every chat turn aggregates against clean structured columns. The trade-off is that an enrichment-prompt change requires re-running the pipeline; the script is resumable (rows with inferred_topic IS NULL are picked up on restart) and idempotent (the ALTER TABLE additions check INFORMATION_SCHEMA before applying), so re-running is operationally cheap.

6.4 The Schema-as-Code Principle

The canonical schema description is the SCHEMA_TEXT constant in tools/sql_tools.py. This string is substituted into the Coder and Code Reviewer prompts at every call (the prompts are re-read from disk rather than cached at module-import time, for prompt-engineering iteration speed). The single source of truth means that a schema change cannot drift between

the SQL, the agent writes, and the database the agent runs against. This ensures that they are always synchronised. The flip side is that schema edits must be made in code, not in the database alone. This is important as the Coder absolutely depends on knowing every column.

7. Memory and Context Management

The system has two memory horizons: a short-term per-session memory held in Streamlit session state, and a long-term cross-session memory persisted to the chat_trace MySQL table.

7.1 Short-Term: Eight-Message Look-Back at the Planner

Short-term memory is exposed to a single agent (the planner) only. The Planner sees the last eight messages from chat history (history[-8:] in agents/planner.py) and produces a resolved_question that bakes the context into a single self-contained string. Every downstream agent (Coder, Output Reviewer, Writer, Viz Coder) reads the resolved_question rather than the raw user message. This architectural choice means that history-aware bugs are contained to the Planner and cannot propagate.

Why eight messages, not four or twenty? There are four reasons for this choice and are documented in the code. First, the empirical observation that follow-ups ("now show me X4", "the same", "give me a chart of that") almost always point back one or two turns; depth beyond four rarely catches new references. Second, the Planner's input budget needs to stay small (max_tokens for output is 400; total prompt is kept at roughly one to two thousand tokens). Third, per-turn cost should stay flat as the session grows. Therefore, capping at eight prevents an unbounded $O(n^2)$ total token cost over a long workday session. Fourth, eight equals 2^3, a generous-but-not-excessive round number that is hard to argue with absent empirical tuning data. The literal eight is a single knob promoted to a module-level constant for easy adjustment.

7.2 The resolved_question Mechanism

Resolution happens exactly once, at the Planner. The Planner's prompt teaches it to recognise two flavors of follow-up. The Vocabulary Resolution flavor maps colloquial phrasing ("stock more", "what's missing") into the technical aggregation the data requires (group by attribute_value, not filter by inferred_topic='Stock Unavailable'). The Hierarchical-Analysis Follow-Up flavor distinguishes Case H (pattern extension like values were discovered, so re-discover at the new scope) from Case V (value carry-forward like the user named the values directly, so carry them forward verbatim). The H-versus-V rule was added after observing the X3

→ X1/X2 failure mode, where the agent kept applying X3-discovered top products to X1 and X2 even though those products were not necessarily the top products at the new stores.

7.3 Long-Term: chat_trace Audit Log

Every chat turn is persisted to a chat_trace MySQL table by tools/trace_logger.py at the end of the orchestrator's run() entry point. The table carries derived columns (path, supervisor_invoked, groundedness_warned, step_count, fail_count, retry_count) plus the full step trace serialised as JSON in steps_json. The schema is denormalised intentionally to make common queries trivial without JSON-path parsing while still preserving full detail. Logging is wrapped in two layers of try/except so a MySQL outage cannot break the chat. The audit log is the foundation on which the team builds offline failure-pattern dashboards and regression analyses.

8. The Agent Roster

Nine LLM agents serve the chat pipeline. Table 7 lists each agent with its model assignment, temperature, max_tokens, and prompt-caching strategy. The two Sonnet 4.6 agents are deliberately tier-matched: the Coder writes the SQL, and the Code Reviewer must be at the same competence tier or it cannot catch the Coder's subtle errors.

Table 7 Agent Roster with Model and Hyper-parameters

Agent	Model	Temp	max_tokens	Prompt Cached?	Role
Planner	Haiku 4.5	0.0	400	Yes	Route + resolve history
Coder	Sonnet 4.6	0.0	900	No (disk re-read)	Chain-of-thought + SQL
Code Reviewer	Sonnet 4.6	0.0	400	No (disk re-read)	Verify SQL technical + reasoning
Output Reviewer	Haiku 4.5	0.0	300	Yes	Semantic fit + viz decision
Writer	Haiku 4.5	0.0	800	Yes	Compose grounded markdown

Agent	Model	Temp	max_tokens	Prompt Cached?	Role
Viz Coder	Haiku 4.5	0.0	900	Yes	Produce matplotlib code
Viz Code Reviewer	Haiku 4.5	0.0	250	Yes	Static safety review
Viz Reviewer	Haiku 4.5 vision	0.0	300	Yes	Multimodal chart review
Supervisor	Haiku 4.5	0.0	400	Yes	Recovery decision-making

8.1 Planner

The Planner is the front door. It classifies each turn into one of three paths [*sql, direct, or clarify*] and emits a resolved_question that bakes in chat history. Routing is, by design, a simpler task than SQL generation; Haiku 4.5 handles it reliably at temp 0. The Planner's prompt teaches it (a) the four-case attribute interpretation tree (Section 7.2), (b) the multi-turn hierarchical-analysis Case H / Case V rule, and (c) the colloquial-vs-literal vocabulary mapping. The Planner is the only agent with chat history visibility; every downstream agent is stateless on history.

8.2 Coder: The SQL-Quality Bottleneck

The Coder converts the resolved natural-language question into a single SELECT SQL statement. Its raw output is plain text with two blocks: a <reasoning>...</reasoning> block containing four named lines (Dimensions, Filters, Aggregation, Completeness) and a <sql_query>...</sql_query> block containing the SQL itself. The reasoning block forces a structural commitment before the model writes SQL which is a chain-of-thought scaffolding pattern.

The Coder runs on Sonnet 4.6 because Haiku 4.5 consistently failed on MySQL window-function syntax and on the non_purchase_type-versus-attribute_value disambiguation that is central to the domain vocabulary. Sonnet 4.6 handles both reliably under the same scaffolding. The Coder's prompt is approximately 381 lines long and contains seven worked examples including a nested-CTE discovery-then-drilldown case that mirrors the Case H follow-up the Planner emits.

The Coder's prompt is deliberately not cached at module-import time. Every call re-reads agents/prompts/coder.txt from disk. This pays a few-hundred-microsecond penalty per call but means that prompt edits take effect on the next chat turn without a full Streamlit restart that is critical for prompt-engineering iteration speed. The same uncached pattern applies to the Code Reviewer's prompt for the same reason; the other agents' prompts are cached because they are edited far less often.

8.3 Code Reviewer

The Code Reviewer judges the Coder's output on two axes: SQL technical correctness, and whether the SQL actually implements the Coder's stated reasoning. It is called twice per attempt: once statically (before sql_executor runs) and again post-execution if the database returned an error. The reviewer at Sonnet 4.6 is intentionally tier-matched with the Coder; a Haiku-tier reviewer cannot catch a Sonnet-tier coder's subtleties. Two worked examples in its prompt illustrate the contradiction cases the reviewer must catch: a Coder that claims "no LIMIT" in reasoning but writes LIMIT 30, and a Coder that lists three dimensions in reasoning but groups by two.

8.4 Output Reviewer

The Output Reviewer runs once sql_executor succeeds. It judges whether the rows semantically answer the question and whether a visualization would help. Its verdict is one of {ok, retry}; its viz_applies field is a separate boolean that gates the entire viz sub-pipeline. The Output Reviewer is the system's defence against the failure mode where SQL ran cleanly but answered the wrong question. For example, a top-reasons query that returned rows from all stores when the user asked about X1 only. Empty results on a clearly answerable broad question are a strong signal of over-filtering and trigger a retry with concrete corrective feedback.

8.5 Writer

The Writer composes the final markdown answer that the user reads. On the sql path the Writer is called inside the hard-block groundedness loop (_write_grounded_answer in the orchestrator); on the direct and clarify paths the Writer is called once. The Writer's prompt forbids referencing UI affordances (the Excel button, the Show SQL expander, the chart image) because those affordances are conditional and rendered by the Streamlit frontend automatically. For e.g. A Writer that says "see the chart below" when no chart was produced creates a contradiction.

The Writer is given a pre-computed Safe Values for Grounding section in its user block. This section, produced by tools.groundedness.safe_derivations_summary, lists every legitimate numeric derivation the groundedness check will accept (grand totals, top-N partial sums, per-group subtotals, per-group row counts). The Writer is instructed to pick numbers from this list rather than compute its own. Computing arithmetic in the orchestrator and feeding the result to the Writer eliminates one of the largest historical sources of groundedness failures: the Writer synthesising a per-group subtotal that happens to miss the candidate set the post-Writer check considers safe.

8.6 Viz Coder

The Viz Coder writes a short matplotlib snippet wrapped in <viz_code>...</viz_code> tags. The snippet is executed by the deterministic viz_generator inside a Python exec sandbox with a whitelisted __builtins__ dictionary. The agent's prompt is explicit about the sandbox: no import statements are allowed (pd, np, plt are pre-loaded), no plt.show() or savefig() calls, and the figure must be set as a variable named fig (plt.gcf() is grabbed as a fallback).

8.7 Viz Code Reviewer

The Viz Code Reviewer performs a static review of the matplotlib code before sandbox execution. The review checks for forbidden identifiers (open, exec, eval, __import__, os, subprocess, requests, urllib, shutil, sys, etc.), the absence of import statements, the presence of titles and axis labels, and the appropriate orientation for bar charts with many categories. Static review catches most safety violations before a single byte of code touches the sandbox.

8.8 Viz Reviewer (Multimodal)

The Viz Reviewer is the only multimodal agent in the system. It sees the rendered PNG plus the resolved question and judges layout, legibility, color coherence, and fit to the question. Its verdict is one of {ok, revise, drop}: revise allows one retry with concrete feedback, while drop is terminal and the chart is abandoned (the text answer still ships). The Viz Reviewer uses the native Anthropic Python SDK rather than aisuite because aisuite's multimodal handling for Anthropic was incomplete at the time of build.

8.9 Supervisor

The Supervisor is invoked only when one of the SQL retry loops has been exhausted. It sees the entire step trace and picks one of four recovery actions: abort_gracefully (explain why

the question can't be answered), retry_with_strategy (propose a fundamentally different approach for one final Coder attempt), ship_partial (ship the best result with a caveat), or ask_user (return a clarifying question). The Supervisor's existence keeps every other agent simple, i.e., they do not need fallback logic because the Supervisor centralises recovery.

8.10 Offline Agent: The Topic Classifier

Outside the chat pipeline lives a tenth agent — the Topic Classifier in data_prep/02_enrich_topics.py. Functionally it is just another LLM agent: prompt with eight worked examples, Haiku 4.5, temperature 0, max_tokens 200, JSON output validated by a Python validate() function with closed-enum and range checks. It is one-time rather than per-turn, so it does not appear in the runtime topology in Figure 1, but architecturally it is a peer to the other nine.

9. Deterministic Capabilities

Also called the tool layer, it has eleven deterministic tools live under tools/. Every tool is pure Python, has a single concern, and is called exclusively by the Orchestrator. Table 8 lists each tool with its purpose and primary failure mode.

Table 8 Deterministic Tool Catalogue

Tool	Module	Purpose	Failure Mode
sql_parser	tools/sql_tools.py	Extract SQL + reasoning from XML tags	Returns ok=False with reason; Coder retries
sql_safety_guard	tools/sql_tools.py	SELECT-only / banned-keyword check	Returns ok=False; Coder retries with feedback
sql_executor	tools/sql_tools.py	Run SQL via mysql.connector, cap 500 rows	Returns {ok:False, error}; Code Reviewer retries

Tool	Module	Purpose	Failure Mode
viz_code_parser	tools/viz_tools.py	Extract Python from <viz_code> tags	Returns ok=False; Viz Coder retries
viz_generator	tools/viz_tools.py	Sandboxed exec of matplotlib code	Returns {ok:False, error}; Viz Coder retries
png_to_base64	tools/viz_tools.py	Encode PNG bytes as base64 ASCII	Vestigial — call_llm_vision does this internally
to_excel	tools/excel_tools.py	openpyxl-backed .xlsx bytes	Returns b''; orchestrator emits to_excel·fail event
groundedness_check	tools/groundedness.py	Regex extraction + set membership	Returns ungrounded list; drives hard-block retry
safe_derivations_summary	tools/groundedness.py	Pre-compute safe values for Writer	Returns empty string when no numeric cols
log_turn / ensure_table	tools/trace_logger.py	Persist chat_trace row + idempotent schema	Try/except swallows everything; never breaks chat
SCHEMA_TEXT / get_schema_for_prompt	tools/sql_tools.py	Canonical schema as a string constant	N/A — read-only

9.1 Why Deterministic Tools, Not Agent Tool-Use

The deliberate non-use of LLM-driven tool selection (the Anthropic tool_use API, MCP servers, OpenAI function calling) was discussed in Section 3.2. From the tool layer's perspective the consequence is concrete: every tool's input is a plain Python value (a string, a list, a dict) and every tool's output is a plain Python value or a small dataclass. Tools never receive raw model text; they receive pre-parsed, pre-validated data. This makes each tool trivially unit-testable and free of any model-specific assumptions.

9.2 Sandboxed Visualization Execution

The viz_generator tool is the only place in the system where untrusted text is executed as Python code. The exec call is given a restricted __builtins__ dictionary that allows only a small whitelist of safe builtins (len, range, sum, min, max, abs, round, sorted, etc.) and explicitly excludes open, exec, eval, __import__, and input. matplotlib is pre-imported with the headless Agg backend, and pandas / numpy are pre-loaded. The trade-off accepted is that this sandbox is dev-grade and for a multi-tenant production deployment the recommended hardening is a separate subprocess with timeout and resource caps. For the present single-user deployment the in-process sandbox is one to two orders of magnitude faster and acceptable.

9.3 Groundedness Check - Why Regex Beats LLM-as-Judge

The groundedness_check tool extracts every numeric token from the Writer's prose and verifies it appears in a candidate set built from the SQL result rows. The candidate set is deliberately broad with seven derivation patterns covering individual cell values, numbers embedded in string labels, per-column grand totals, per-row column percentages, per-group subtotals, per-group row counts, and top-N partial sums for $N \in \{2,3,4,5\}$. A 0.5-unit rounding tolerance handles common rounding ($23.5\% \approx 24\%$).

Choosing regex over an LLM-as-judge alternative was deliberate. A regex check is deterministic, free, and trivially explainable to a stakeholder asking "how do you know this number is right?" An LLM-as-judge would be stochastic, would add another API call per turn, and would require its own evaluation harness to know that the judge is correctly judging. The trade-off accepted is that the regex check cannot verify semantic attribution i.e., it can confirm 41 appears somewhere in the rows but cannot confirm 41 was attributed to the right group. The Output Reviewer is the agent that catches misattribution; the layering is intentional.

10. The Agentic Workflow End-to-End

Figure 2 renders the temporal flow of one SQL-path chat turn from the user's question to the rendered answer. The workflow tree has eight to twelve nodes from input to output because every node either does specialized work or guards against a specific failure mode.

10.1 Path Selection by the Planner

Every turn begins with the Planner. If the user said "hi" or "what does X mean?", the Planner returns path=direct and the Writer composes a conversational answer with no database access. If the user said "tell me about my stores" with no history, the Planner returns path=clarify and the orchestrator returns the clarifying question to the user. Otherwise, the Planner returns path=sql and the SQL pipeline begins.

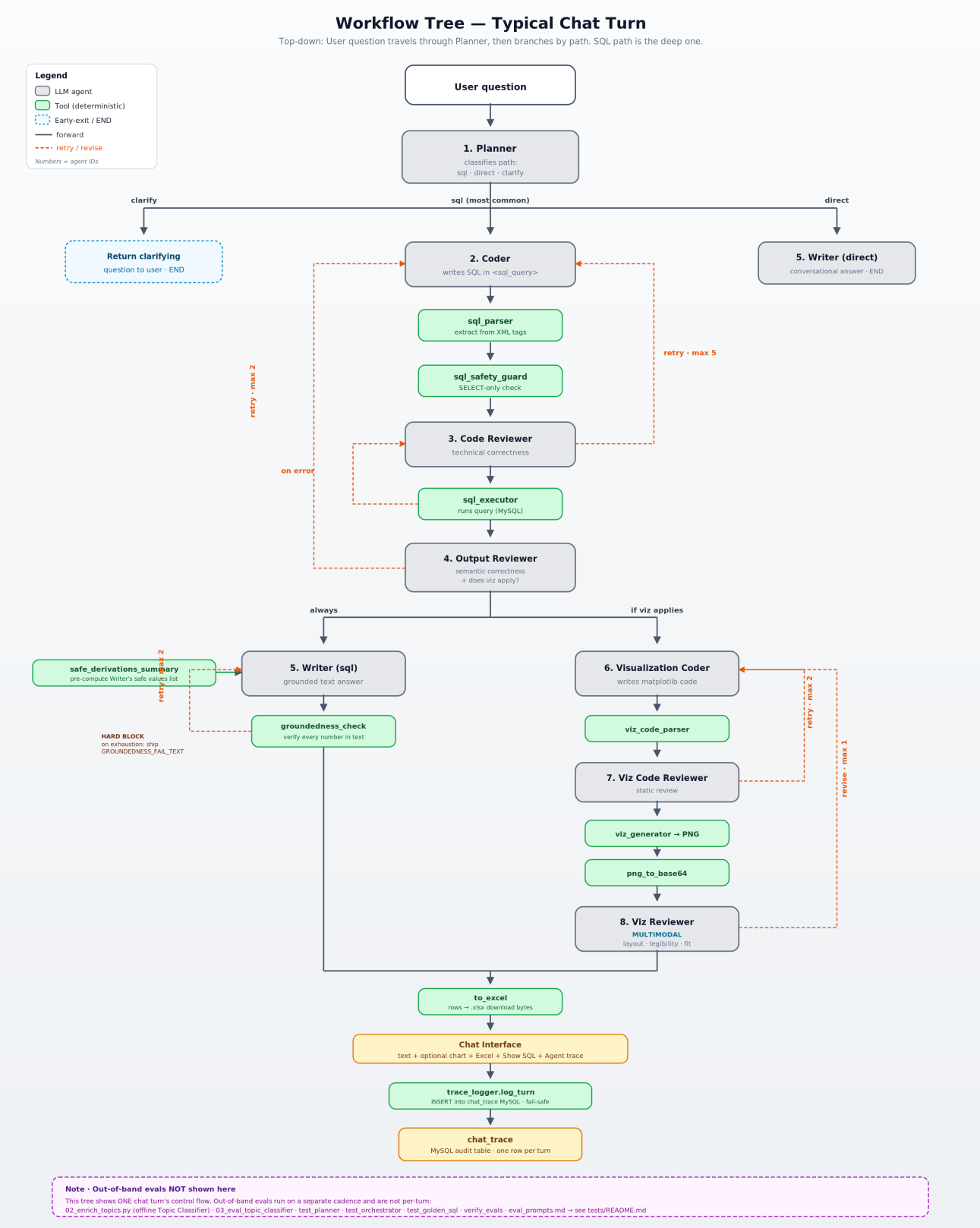
10.2 The Inner SQL Loop

The Coder writes an attempt; sql_parser extracts the reasoning and SQL blocks; sql_safety_guard checks SELECT-only / banned-keyword / multi-statement; the Code Reviewer verifies technical correctness and reasoning-versus-SQL consistency; sql_executor runs the SQL against MySQL with a 500-row cap. On any of parse failure, safety failure, code-review retry, or execution error, the loop circles back to the Coder with reviewer feedback as a hint and the previous SQL marked as rejected. The loop runs up to five times before escalating to the Supervisor.

10.3 The Outer Output-Review Loop

Once sql_executor succeeds, the Output Reviewer judges whether the rows answer the question. If verdict=retry, the loop circles back to the Coder with semantic feedback as a hint. The Output Reviewer loop runs up to two times before escalating to the Supervisor. This loop is the system's defence against SQL that ran cleanly but answered the wrong question.

Figure 2 Temporal Agentic Workflow — one SQL-path chat turn from input to rendered answer



10.4 The Writer ↔ Groundedness Hard-Block Loop

After the Output Reviewer accepts the result, _write_grounded_answer runs the Writer up to two times. Each Writer attempt's output is fed to groundedness_check. If any number in the prose does not reconcile against the candidate set, the loop circles back to the Writer with the unmatched-numbers feedback string. If after two attempts the numbers still do not reconcile, the orchestrator does not ship the unverified text: it substitutes a fixed GROUNDEDNESS_FAIL_TEXT message that explicitly states "I couldn't produce verifiable numbers" and points the user at the Show SQL and Excel download surfaces for direct inspection.

10.5 The Viz Sub-Pipeline (Conditional)

If the Output Reviewer flagged viz_applies=true and at least one row was returned, the orchestrator runs the viz sub-pipeline in sequence (not in parallel) with the Writer. The Viz Coder writes a matplotlib snippet; the Viz Code Reviewer statically reviews it; viz_generator executes it in the sandbox; the Viz Reviewer multimodally inspects the rendered PNG. The static-review loop runs up to two times; the multimodal-review revise loop runs up to one time. If either budget exhausts, the chart is silently dropped and the text answer ships without it.

10.6 Excel Attachment

Whenever row_count > 0, the orchestrator calls to_excel via the _attach_excel helper. The helper handles the three SQL-path return sites: happy path, ship_partial Supervisor branch, retry_with_strategy Supervisor branch, so every grounded SQL answer carries the same Excel download affordance. If to_excel fails (typically because openpyxl is missing), the orchestrator emits a to_excel·fail step event so the failure is visible in the trace.

10.7 Persistence

Finally, the orchestrator's run() entry point calls trace_logger.log_turn after _run_impl returns. This INSERTs one row into chat_trace with derived metadata plus the full step JSON. The call is wrapped in try/except so a MySQL outage cannot affect the response that has already been returned to the user.

11. Reflection, Self-Critique, and Cross-Review

Reflection in this system is performed by dedicated reviewer agents whose only job is to critique the output of the corresponding reasoning agent. Each reviewer's verdict is a closed

enum and each reviewer's feedback is a concise concrete suggestion. The reasoning-versus-reflection split was chosen over a single self-critiquing agent for two reasons. First, prompting one agent to both produce and critique its own output dilutes the system prompt and produces measurably worse criticism. Second, separating the two creates a clean retry-budget boundary. The reviewer's verdict is what determines whether the loop continues or terminates, and that decision belongs in its own agent with its own narrow prompt.

11.1 Static vs Post-Execution Review (Code Reviewer)

The Code Reviewer is called twice per SQL attempt. Once statically before execution, and once after a database error. The same function review() handles both passes; the only differences are the execution_error string passed in the second call. This dual-call pattern catches two distinct error classes: at static time it catches structural mistakes (wrong column names, missing GROUP BY, ground_truth_topic referenced in production query, reasoning-versus-SQL contradictions); at post-execution time it catches data-driven mistakes (typo'd column names not flagged statically, syntax that the regex parser accepted but MySQL rejected, semantic mismatches that only manifest at execution time).

11.2 Reasoning-Versus-SQL Verification

The Code Reviewer's most consequential check is the reasoning-versus-SQL verification. The Coder is required to emit a <reasoning> block with four named lines: *Dimensions, Filters, Aggregation, Completeness*. The Code Reviewer's prompt instructs it to verify the SQL honors each line. If the reasoning says "three dimensions" but the SQL groups by two, that is a retry. If the reasoning says "no LIMIT" but the SQL has LIMIT 30, that is a retry. If the reasoning lists a filter that the SQL omits, that is a retry. This single check eliminates the most damaging Coder failure mode, plausible-looking SQL that doesn't match intent, that the static SQL review on its own cannot detect.

11.3 Multimodal Reflection on Charts

The Viz Reviewer is multimodal and is sent the rendered PNG along with the resolved question. The Viz Code Reviewer (static) cannot catch a chart whose code ran but produced a confusing or mislabelled image. The Viz Reviewer can. Its verdict triplet {ok, revise, drop} is richer than the binary verdicts of other reviewers because the multimodal review can distinguish a small fixable issue (axis labels overlap → revise) from an unsalvageable error (wrong data plotted → drop).

11.4 Why Not a Single "Critic" Agent at the End

A common alternative pattern is a single critic agent at the end of the pipeline that judges everything. The system rejects this pattern because the critic at the end is too late and by the time it fires, the system has already paid for the Coder, sql_executor, the Writer, and potentially the viz pipeline. Critique at each stage means rejection happens at the cheapest point in the pipeline at which the error is detectable, which is the right place for a bounded retry budget to be enforced.

12. Orchestration, Retry Budgets, and Recovery

The Orchestrator (agents/orchestrator.py, 679 lines) is the only stateful actor in the system. Its public entry is run(user_message, history), which wraps the internal _run_impl plus a final call to trace_logger.log_turn. All retry budgets are encoded as module-level constants at the top of the file.

12.1 Retry Budgets

Table 9 Retry Budgets and Escalation Behaviour

Loop	Constant	Max	Exhaustion Behaviour
Coder ↔ Code Reviewer ↔ sql_executor	MAX_CODE_REVIEW_RETRIES	5	Escalate to Supervisor
Output Reviewer ↔ Coder	MAX_OUTPUT_REVIEW_RETRIES	2	Escalate to Supervisor
Writer ↔ groundedness_check	MAX_GROUNDEDNESS_RETRIES	2	Ship GROUNDEDNESS_FAIL_TEXT (no Supervisor)
Viz Coder ↔ Viz Code Reviewer	MAX_VIZ_CODE_RETRIES	2	Drop chart; ship text only (no Supervisor)
Viz Reviewer revise round	MAX_VIZ_REVISE	1	Ship last PNG, or drop if verdict was 'drop'

12.2 Why These Specific Budget Numbers

The five-attempt budget on the Coder ↔ Code Reviewer loop is generous enough to recover from at least two compounded mistakes (a typo plus a missing GROUP BY plus a LIMIT removal) but tight enough to bound worst-case turn cost. The two-attempt budget on the Output Reviewer outer loop is intentionally tighter because semantic rejections are usually decisive. If the result genuinely fails to answer the question, a third attempt with the same Coder is unlikely to help; better to escalate to the Supervisor for a strategy change. The two-attempt budget on the Writer ↔ groundedness loop is similarly tight because if the Writer can't ground numbers in two attempts, a third attempt rarely helps, and the cost of shipping wrong numbers exceeds the cost of shipping a polite refusal.

12.3 The Supervisor's Four Recovery Actions

When a SQL-path retry budget is exhausted, _supervisor_fallback invokes the Supervisor with the full step trace serialised as a list of dicts. The Supervisor returns one of four actions: abort_gracefully (the question is genuinely unanswerable; ship a polite explanation), retry_with_strategy (the Coder is stuck; here is a fundamentally different SQL approach to try one more time), ship_partial (we got a useful result, ship it with a caveat), or ask_user (the question is ambiguous; here is a focused clarifying question). The retry_with_strategy action is a one-shot.

12.4 Hard-Block vs Soft-Block - Why Two Different Exhaustion Modes

Three of the five retry loops escalate to the Supervisor for recovery (Coder loop, Output Reviewer loop, the Supervisor's own one-shot retry). Two loops do not (Writer groundedness, Viz pipeline). The choice is deliberate. For the SQL loops, a strategy change can plausibly help. Different SQL might answer the question correctly. For the Writer-groundedness loop, the failure is local to the prose-composition step and a strategy change does not help; the right answer is to refuse with a fixed message and surface the underlying data via the Show SQL and Excel affordances. For the Viz pipeline, the chart is supplementary; if it cannot be produced, the text answer still ships and the user can ask for a chart again.

13. Prompt Engineering Choices

Prompts live as plain .txt files in agents/prompts/. Storing prompts as files are clean, prompts are editable without touching code, and a future prompt-versioning / A-B testing layer can hook in

at the load_prompt function without touching every agent. Table 10 lists every prompt file with its size.

Table 10 Prompt File Catalogue

Prompt File	Lines	Owner Agent
planner.txt	165	Planner
coder.txt	412	Coder
code_reviewer.txt	100	Code Reviewer
output_reviewer.txt	111	Output Reviewer
writer.txt	132	Writer
viz_coder.txt	80	Viz Coder
viz_code_reviewer.txt	35	Viz Code Reviewer
viz_reviewer.txt	37	Viz Reviewer
supervisor.txt	37	Supervisor

13.1 The Three Coordinated CoT Scaffolds

Chain-of-thought scaffolding in this system is not a single prompt trick but a triplet of coordinated mechanisms. The first is the Coder's <reasoning> block (Dimensions / Filters / Aggregation / Completeness) that forces a structural commitment before SQL. The second is the safe-derivations list injected into the Writer's user block, which takes arithmetic off the LLM and replaces it with a curated candidate set. The third is the Code Reviewer's reasoning-verification rule, which closes the loop by checking that the Coder's SQL actually implements the reasoning the Coder stated.

Each of the three on its own buys some quality; together they moved the system from "occasionally wrong" to "reliably right on the first attempt" on the manual eval suite. Removing any one of the three causes the system to regress; this was empirically observed during prompt-engineering iteration.

13.1.1 Example of all Three Scaffolds in One Turn

Consider a representative compound merchandising question: *"Which product should I stock more at X2, and what attribute is most needed for it?"* This question is structurally non-trivial. It carries two dimensions (**product_looking_for** and **attribute_value**), one filter (**store_code = 'X2'**), and is a planning-oriented question. The Writer needs every group/value cell to compose a useful answer, not a top-N truncation. The three scaffolds work in sequence to convert this question into a correct, grounded, and verified answer.

Scaffold 1 — the Coder's <reasoning> block forces a structural commitment before any SQL is written. For the question above, the Coder produces:

```
<reasoning>
- Dimensions: TWO — product_looking_for AND attribute_value.
- Filters: store_code='X2', attribute_value IS NOT NULL.
- Aggregation: COUNT(*) grouped by (product_looking_for, attribute_value).
- Completeness: planning question, return ALL rows. NO LIMIT.
</reasoning>

<sql_query>
SELECT product_looking_for, attribute_value, COUNT(*) AS n
FROM non_purchasers_feedback
WHERE store_code = 'X2' AND attribute_value IS NOT NULL
GROUP BY product_looking_for, attribute_value
ORDER BY n DESC
</sql_query>
```

Once the four reasoning lines are committed in text, the SQL is essentially constrained to follow them. The model's own consistency bias makes it linguistically jarring to write LIMIT 30 immediately after stating "NO LIMIT" in the reasoning. The scaffold turns implicit intent into explicit, auditable structure.

Scaffold 3 — the Code Reviewer's reasoning-verification rule closes the loop on Scaffold 1. The reviewer reads both the <reasoning> block and the SQL and verifies one against the other. If the Coder had instead emitted SQL with only one dimension (GROUP BY attribute_value) or with LIMIT 30 despite the reasoning stating no LIMIT, the reviewer would issue a retry verdict with concrete feedback such as: *"Reasoning lists TWO dimensions but SQL only groups by attribute_value; add product_looking_for to SELECT and GROUP BY."* This catches plausible-looking SQL that quietly contradicts its own plan.

Scaffold 2 — the safe-derivations list takes arithmetic off the Writer. After the SQL runs and returns the result rows, the orchestrator runs safe_derivations_summary in plain Python and injects a Markdown block into the Writer's user prompt:

```
## Safe values for grounding (pre-computed — pick from these)
  - Grand total of `n` across all rows: **88**
  - Top-3 partial sum of `n` (sorted desc): **43**
  - `product_looking_for`='Necklace': subtotal of `n` = **43** across **3**
row(s)
  - `product_looking_for`='Bangles':  subtotal of `n` = **16** across **2**
row(s)
  - `attribute_value`='rose gold':    subtotal of `n` = **18** across **1**
row(s)
  ...
```

Given this list, the Writer composes:

"Necklace has the highest non-purchase volume at X2 with **43 feedbacks** *across 3 distinct attributes. Within Necklace, customers most often asked for* **rose gold (18 requests)**, **temple design (14)**, *and* **star design (11)**. *The actionable takeaway: prioritise restocking rose-gold finishes and temple-design pieces in the Necklace category at X2."*

Every number cited [43, 18, 14, 11] is either a per-group subtotal lifted directly from the safe-derivations list or an individual cell value from the result rows. No arithmetic was performed by the LLM. The post-Writer groundedness check verifies each numeric token against the same candidate set the safe-derivations list was built from and ships the answer.

The three scaffolds, summarised in the context of this turn. Scaffold 1 prevented the Coder from missing the second dimension. Scaffold 3 would have caught the contradiction had Scaffold 1 been honored in reasoning but ignored in SQL. Scaffold 2 prevented the Writer from inventing 42 when the correct subtotal was 43. Each scaffold guards a distinct failure surface, and the three together cover the full path from natural-language question to grounded, citable prose.

13.2 Few-Shot Examples — How Many and Why

The Planner prompt carries nine worked examples covering the colloquial-vs-literal vocabulary distinction (three examples), the Case H vs Case V follow-up rule (two examples), direct path (two examples), comparison and chart cases (two examples). The Coder prompt carries seven worked examples including a nested-CTE discovery-then-drilldown case. The Topic Classifier prompt carries eight worked examples covering each attribute_type plus stock-style NULL values and Others-style NULL values. The Code Reviewer prompt carries six worked examples covering the reasoning-verification patterns. The number of few-shot examples per prompt is governed by the breadth of decision space the agent must cover; agents with narrower decision spaces (Viz Code Reviewer, Supervisor) need correspondingly fewer.

13.3 Closed Vocabulary in Every Prompt

Every reviewer prompt enumerates the allowed verdicts in the output schema ("ok" or "retry"; the Viz Reviewer adds "drop"). The Supervisor enumerates the four actions. The Planner enumerates the three paths. This pattern of "the JSON output must have a key whose value is one of these N strings" is the single most reliable LLM-output discipline available, and is enforced both at the prompt level and in code by Python checks on the parsed JSON.

14. Evaluation Strategy and Metrics

Evaluation is layered across two horizons: **runtime in-band evaluation** that runs every turn, and **out-of-band evaluation that runs on a cadence**. The two horizons cover different failure modes. In-band catches per-turn errors before they reach the user, out-of-band catches regressions when prompts or models change.

14.1 In-Band: The Three Runtime Rings

Ring 1 is the deterministic guards: sql_parser, sql_safety_guard, viz_code_parser, the viz_generator sandbox, groundedness_check, and the closed-enum validators on every agent's structured output. These guards cannot lie because they are pure Python.

Ring 2 is the cross-reviewing LLM agents: Code Reviewer, Output Reviewer, Viz Code Reviewer, Viz Reviewer. These are LLMs and can occasionally be wrong, but they catch the bulk of subtle errors the deterministic guards cannot see.

Ring 3 is the retry loops with the Supervisor escape hatch: every reviewer's veto bounces back to the relevant reasoning agent with feedback as a hint; bounded retries protect against infinite loops; the Supervisor recovers when the SQL loops exhaust.

14.2 Out-of-Band: Four Eval Suites

Four out-of-band evaluation suites live under tests/ and data_prep/. Each plays a distinct role in the trust-but-verify story; running all four after a prompt or model change is the only reliable way to confirm no regression has occurred.

- **tests/verify_evals.py** — 7-check integration smoke test confirming groundedness behaviors and trace_logger round-trip.
- **tests/test_golden_sql.py + golden_sql_dataset.jsonl** — 26-entry automated Q-to-SQL regression suite. Shape check, CI-style exit codes.

- **tests/eval_prompts.md** — 15 manual prompts at three difficulty tiers for ground-truth answer-correctness comparison against Excel pivot tables.
- **data_prep/03_eval_topic_classifier.py** — Offline Topic Classifier accuracy and confusion-matrix evaluation.

14.3 Golden SQL Dataset — Coverage

Table 11 Golden SQL Dataset — Coverage Summary (26 entries total)

Category	Entries	Examples
Single-store top-reason	3	001, 003, 010
Cross-store aggregation	2	002, 025
Two-store comparison	3	005, 021, 012
Attribute-value drill-down	3	006, 007, 019
Compound multi-dimension	2	008, 022
Direct path (greetings, definitions)	3	013, 014, 015
Clarify path (vague questions)	2	016, 017
Chart-request preservation	1	018
Colloquial-vs-literal vocabulary	2	019, 020
Topic-filtered ranking	2	004, 011
Loose product filters	2	023, 024

The golden dataset is intentionally a shape check rather than an answer-correctness check. Each entry asserts (a) the Planner picks the expected path, (b) the generated SQL contains the required tokens and avoids the forbidden ones, and (c) the row count falls within bounds. It does not assert that the actual numbers in the answer are correct, but that is the job of the manual eval suite (eval_prompts.md, 15 prompts at three difficulty tiers).

14.4 Topic Classifier Evaluation

data_prep/03_eval_topic_classifier.py compares the LLM-classified inferred_topic against the synthetic ground_truth_topic for every enriched row. It reports overall accuracy, per-

topic precision, recall, F1, the full confusion matrix, and average topic_confidence on correct versus incorrect predictions. The four equations below define the metrics.

Equation 1 Precision per topic t

$$\text{Precision}(t) = TP(t) / (TP(t) + FP(t))$$

Equation 2 Recall per topic t

$$\text{Recall}(t) = TP(t) / (TP(t) + FN(t))$$

Equation 3 F1 per topic t

$$F1(t) = 2 \cdot \text{Precision}(t) \cdot \text{Recall}(t) / (\text{Precision}(t) + \text{Recall}(t))$$

Equation 4 Overall accuracy

$$\text{Accuracy} = (\Sigma \text{ over } t \text{ of } TP(t)) / N\_total$$

Here TP(t), FP(t), FN(t) are true positives, false positives, and false negatives for topic t respectively, and N_total is the total number of evaluated rows. The script also prints the lowest-confidence mistakes for spot-checking. A regression often manifests as a small number of high-confidence wrong predictions, which is more informative than the overall accuracy number alone.

14.5 Why Both a Shape Check and an Answer-Correctness Check

The distinction between test_golden_sql.py (automated shape check) and eval_prompts.md (manual answer-correctness check) is deliberate. A prompt change that subtly alters the row count an aggregation returns will not be caught by the shape check if the row count still falls within the asserted bounds; it requires the manual numeric comparison to catch. Conversely, a refactor that breaks the Planner's path-routing will not be caught by the manual eval (which compares numbers) but is immediately caught by the shape check (which asserts path). Both layers are needed.

15. Validations and Safety Guards

Safety in this system is multi-layered and exists at three levels: deterministic guards in the tools, closed-enum validation on every LLM output, and the regex-based numeric groundedness check on the Writer's prose.

15.1 sql_safety_guard

The sql_safety_guard tool runs after sql_parser and before sql_executor. It enforces three rules. First, the SQL must begin with SELECT or WITH (the latter for CTE-style discovery-then-drilldown queries). Second, a banned-keyword regex rejects insert, update, delete, drop, alter, truncate, rename, create, grant, revoke, replace, merge, call, execute, exec, load, outfile, and into outfile. Third, multi-statement SQL is rejected (a semicolon followed by any non-whitespace character). Any rejection bounces back to the Coder with the rejection reason as feedback. This is the SQL-injection mitigation surface.

15.2 Closed-Enum Validation

Every LLM agent emitting a structured field has that field's allowed values enumerated in Python. The Planner's path is checked against {sql, clarify, direct}. The Code Reviewer's verdict is checked against {ok, retry}. The Output Reviewer adds the viz_applies boolean. The Viz Reviewer's verdict is {ok, revise, drop}. The Supervisor's action is checked against {abort_gracefully, retry_with_strategy, ship_partial, ask_user}. Out-of-enum output raises ValueError, which the Orchestrator catches and counts against the relevant retry budget.

15.3 Sandboxed Visualization Execution

The viz_generator sandbox restricts __builtins__ to a fourteen-item whitelist (len, range, enumerate, zip, sum, min, max, abs, round, sorted, reversed, list, dict, set, tuple, str, int, float, bool, isinstance, print, True, False, None) and explicitly omits open, exec, eval, __import__, and input. matplotlib uses the headless Agg backend; pandas and numpy are pre-loaded; df is built from the SQL result rows. The sandbox is documented as dev-grade and for production multi-tenant deployment would require a separate-subprocess isolation with timeout and resource caps.

15.4 The Groundedness Check

For a quantitative business-analytics use case, numeric hallucination is the most damaging failure mode the system can produce: a fabricated count in a stock-planning answer becomes a wrong purchase order, a misallocated reorder, or a misdirected merchandising decision. The downstream cost of a single wrong number is deeply asymmetric. It dwarfs the engineering cost of catching it. Investing a deterministic, hard-blocking guard whose only job is to verify every number the Writer cites against the underlying data is therefore the highest-leverage quality intervention available in the system. It is, in spirit, what unit tests are to a

software library: cheap to run, cheap to maintain, and structurally incapable of being talked into letting a bug through.

The groundedness_check tool is the only validator that operates on the final user-facing prose. It runs as a hard-block inside the Writer's retry loop: every numeric token in the prose must reconcile against the candidate set derived from the SQL result rows. The candidate set includes individual cells, numbers embedded in string labels, per-column grand totals, per-row column percentages, per-group subtotals, per-group row counts, and top-N partial sums for N ∈ {2, 3, 4, 5}, with a 0.5-unit rounding tolerance. Numeric tokens with absolute value below 2.0 are ignored to avoid false positives on years and IDs. The lookahead (?!\w) in the extraction regex skips digits inside unit-suffixed labels like "25k" or "10inch".

15.5 Schema-Level Vocabulary Discipline

A safety guard often overlooked is naming. The column originally named attribute_type was renamed non_purchase_type because the word "attributes" naturally collided with both columns in user phrasing. A customer asking "which attributes should we stock more" might be referring to either column. The rename was a small change that prevented an entire class of vocabulary-disambiguation errors in the Coder, and is documented inline in the Coder's prompt with a non-negotiable header rule that explicitly states the resolution.

16. Traceability and Observability

Observability is a first-class concern. Every chat turn emits a stream of StepEvent objects with four fields: agent (which agent or tool acted), status (start, ok, retry, or fail), summary (a one-line human-readable description), and detail (a dict carrying agent-specific structured data that includes the Coder's reasoning, the SQL text, the SQL executor's row count, the Output Reviewer's viz_applies flag, the groundedness check's ungrounded number list).

16.1 The chat_trace Audit Table

Every turn is persisted to a chat_trace MySQL table by tools/trace_logger.py. Table 12 lists the chat_trace schema. Common queries (supervisor invocation rate, average retry count per path, groundedness warning rate, most-expensive failures) are pre-baked in the system-overview documentation.

Table 12 chat_trace Schema — one row per chat turn

Column	Type	Purpose
trace_id	BIGINT PK	Auto-incrementing turn identifier
created_at	DATETIME	Insertion timestamp
user_message	TEXT	Raw user turn
path	VARCHAR(16)	sql / direct / clarify
resolved_question	TEXT	Planner's context-resolved question
final_sql	TEXT	SQL that ran (NULL on direct/clarify)
has_chart	BOOLEAN	Whether a chart was rendered
has_excel	BOOLEAN	Whether an Excel download was attached
supervisor_invoked	BOOLEAN	Whether the Supervisor was reached
groundedness_warned	BOOLEAN	Whether groundedness failed (hard block)
step_count	INT	Number of StepEvents in this turn
fail_count	INT	Number of steps with status=fail
retry_count	INT	Number of steps with status=retry
answer_text	MEDIUMTEXT	Final user-facing markdown
steps_json	MEDIUMTEXT	Full step trace as JSON

16.2 Indexing for Common Queries

chat_trace carries four indexes: idx_created on created_at for time-ordered queries, idx_path on path for per-path aggregation ("average retry count on the sql path"), idx_supervisor on supervisor_invoked for failure-pattern queries, and idx_grounded on groundedness_warned for triaging hard-blocked answers. The denormalised schema (derived fields plus a JSON column) was chosen over a normalised step-event table because the common query patterns are "give

me all turns where X happened" rather than "give me every step in turn Y". The JSON column makes the latter still possible via JSON-path functions when needed.

16.3 Fail-Safe Logging

log_turn is wrapped in two layers of try/except. The inner layer is inside log_turn itself; the outer layer is in the orchestrator's run() entry point. A MySQL outage during logging prints a single line to stderr and returns; the chat response that has already been computed is returned to the user unchanged. This is non-negotiable: observability infrastructure cannot affect the system it observes.

16.4 In-UI Trace Surfacing

Every Streamlit chat response carries four expandable panels: the answer markdown is rendered inline; "Show SQL" expands to reveal the executed SQL; "Show chart code" expands to reveal the matplotlib snippet; and "How I got this answer" expands the full StepEvent trace including the Coder's reasoning block. This direct exposure means a sceptical user can audit the system without leaving the chat surface. They do not need access to the chat_trace table to understand why the answer is what it is.

17. Ethical Guidelines, Privacy, and Responsible AI

Ethics in this system manifests in three concrete commitments: refuse to ship unverified numbers, expose every step of reasoning to the user, and never let an LLM speak with authority outside its competence. Each of the three is implemented as a runtime mechanism rather than a policy on a wiki page.

17.1 Refuse Rather Than Mislead

The Writer ↔ groundedness hard-block loop is the system's most consequential ethical stance. When the Writer cannot produce prose whose every numeric token reconciles against the candidate set after two attempts, the system ships an explicit refusal (GROUNDEDNESS_FAIL_TEXT), that tells the user "*I couldn't produce an answer with verifiable numbers*" and points them at the Show SQL and Excel download surfaces. A business user is far better served by an honest refusal than by polished prose with subtly wrong numbers. The trade-off accepted is the small fraction of turns on which a marginal grounding mismatch hard-blocks an answer that a human would judge essentially correct; the design treats this false-positive cost as smaller than the false-negative cost of misleading numbers.

17.2 Radical Transparency

Every chat response is accompanied by the SQL that ran, the chart code that produced the chart, and the full agent trace. There is no "black box." A user who distrusts an answer can read the SQL, run it themselves against the Excel export, and confirm the chain of reasoning. This contrasts with many chat assistants that expose only the final answer; the difference is doctrinal. The system is built for a stakeholder audience that must defend its merchandising decisions in front of inventory teams and store managers, and "the AI said so" is not an acceptable defence.

17.3 Constraining Authority

The Writer's prompt explicitly forbids referencing UI affordances the user might or might not see (the Excel button, the Show SQL expander, the chart image). This is partly a UI-consistency rule but partly an ethical one: the Writer must not promise the user something that may not appear. By keeping the Writer's claims narrow ("based on the data, Necklace shows 41 design issues at X2") the system avoids the failure mode where the model gestures at evidence that does not exist.

17.4 Privacy and PII Handling

The **non_purchasers_feedback** table carries two columns that constitute personally identifiable information (PII): **customer_name** and **customer_email**. Both are queryable today and no automatic redaction layer is in place. Empirically, however, the chat surface rarely returns PII in its answers, a fact that is comforting on first inspection but is the result of **incidental behavioural protections** rather than designed structural safeguards. The distinction matters because incidental protection degrades sharply under adversarial or unusual queries, whereas designed protection is robust to both. The remainder of this subsection documents what keeps PII out of typical answers today, why those mechanisms are not sufficient, and what designed protection would look like.

The four incidental mechanisms in effect today. Four overlapping behavioural mechanisms keep PII out of typical chat responses, none of them deliberate. **First,** the Coder's prompt corpus is entirely aggregation based. The seven worked examples in coder.txt and the 26 entries in golden_sql_dataset.jsonl all show GROUP BY / COUNT(*) / top-N patterns, and no example anywhere references customer_name or customer_email in a SELECT clause. Few-shot pattern-matching is one of the strongest behavioural levers on a large language model, so the Coder defaults to aggregation even on ambiguous questions. **Second,** the Planner's resolved_question rewriting biases toward aggregation phrasing. The Planner is taught by

example to translate colloquial questions into structured aggregation queries, and none of its worked examples resolve a question into a row-list query that would surface individuals. **Third,** the Writer's prompt instructs grouped-summary composition. Its template ("lead with the dominant value of the first dimension," "show a small markdown table," "end with a 1-sentence actionable takeaway") has no scaffolding for listing individual rows with customer names, so even on the rare occasions when result rows happen to contain PII, the Writer's compositional pattern does not naturally pull those values into the output. **Fourth,** Anthropic's RLHF training provides a thin, incidental backstop. Claude has been safety-trained to avoid casual exposure of personal information, which adds a faint preference against gratuitously including PII; this is not a robust safeguard (it loses out to a clear contrary instruction in the system prompt) but it does reduce the probability of accidental leakage on borderline cases.

Why these protections are incidental, not designed. Each of the four mechanisms above is a *default behaviour that happens to be safe for the typical query distribution*, not a *guard that prevents PII from leaving the data layer*. A directly phrased query exposes the gap. A user asking *"list the names and emails of customers who complained about Size Unavailable at X1"* would proceed through the Planner (path=sql, resolved into a row-list query), the Coder (writing SELECT customer_name, customer_email, reason_for_non_purchase ... since nothing forbids those columns), the safety guard (which has no PII rule), the Code Reviewer (which has no PII rule), sql_executor (which returns the columns requested), and the Writer (which, with mild reluctance from Claude's safety training, would most likely compose the answer including the names). The system in its current form cannot reliably refuse such a query because nothing has been designed to refuse it.

Designed protection — future work. Two complementary changes would move the protection from incidental to designed, with each implementable in roughly a hundred lines of code and verifiable via a new entry in tests/verify_evals.py:

- **A PII-scrubbing wrapper around sql_executor.** A short Python function that removes customer_name, customer_email, and any future PII columns from every result row before it is returned to the orchestrator. The Coder, Output Reviewer, and Writer never see the PII columns regardless of what SQL the Coder writes. This is a deterministic structural guarantee, unit-testable, and unaffected by future prompt or model changes.
- **A Writer-prompt rule that forbids citing individual customers.** Even with the executor-level scrub in place, an explicit prompt rule provides a defence-in-depth layer that protects against any future change to the scrub logic. The rule fits in three or four lines of

writer.txt and is enforceable in tests by sending the Writer a synthetic row set that contains plausibly-PII-like strings and asserting they do not appear in the output.

Table 13 below summarises the difference between today's incidental protection and the recommended designed protection.

Table 13 Incidental versus Designed PII Protection

Property	Incidental protection (today)	Designed protection (recommended)
Mechanism	Prompts and few-shot examples bias the agents toward aggregation; PII columns are never named in the corpus.	sql_executor wrapper scrubs PII columns from result rows; Writer-prompt rule forbids citing individuals.
Determinism	Probabilistic — depends on prompt phrasing, model temperature, and example coverage.	Deterministic — PII never leaves the data layer regardless of the SQL the Coder writes.
Resilience to adversarial input	Brittle — a directly phrased PII request will likely succeed.	Robust — the agents never receive PII columns, so the SQL itself cannot exfiltrate them.
Auditability	Hard — cannot prove no leakage without inspecting every chat_trace row.	Easy — the scrub function has a unit test in verify_evals.py.

When the upgrade becomes mandatory. The decision to ship without designed PII protection is acceptable in the current internal-only deployment context, where the user population is the merchandising team itself and questions are aggregation-shaped by professional habit. The decision must be revisited before any of the following: external user access (any non-employee being granted chat permission); wider internal access (any role outside merchandising or inventory being granted permission); a regulatory audit that requires demonstrable PII handling; or a use-case expansion that adds questions about individual customers (for example, loyalty-program analysis or single-customer journey reviews). At any of

those triggers, the recommended hardening above should be implemented and verified by a new entry in tests/verify_evals.py before access is granted.

17.5 Bias and Fairness Considerations

The Topic Classifier is the system's most consequential bias surface. A topic-classifier that systematically under-represents (for example) Color/Finish Mismatch complaints in one store would cause merchandising decisions to silently under-prioritise that store's color inventory. The mitigation is the offline 03_eval_topic_classifier.py harness, which reports per-topic precision, recall, F1, and confusion matrix against the synthetic ground_truth_topic anchor. Per-topic F1 below a threshold should trigger a prompt revision before the affected topic's predictions are used downstream. The threshold is currently a manual judgement; making it an automated CI gate is a recommended near-term hardening.

17.6 Use of Synthetic Data and Disclosure

The 1,200 row seed dataset is synthetic, generated by data_prep/ 01_generate_feedback_data_mysql.py with random.seed(42) for reproducibility. The synthetic origin is documented and the ground_truth_topic column exists only because the rows are synthetic — it would not exist in real production data. Replacing the synthetic data with real customer feedback in production will require dropping the ground_truth_topic column from the schema, adapting the eval harness to operate against a held-out human-labeled sample, and ensuring that all customer-identifying fields are subject to the privacy hardening described above.

18. Performance, Cost, and Limitations

18.1 Latency Per Turn

SQL-path happy case (one Coder attempt, one Output Reviewer pass, one Writer attempt, no chart): approximately 15 to 20 seconds end-to-end, dominated by the Coder (Sonnet 4.6 with a large prompt and chain-of-thought output), Code Reviewer (Sonnet 4.6, smaller output), and Writer (Haiku 4.5).

SQL-path with retries: 30 to 60 seconds when one or two retry attempts occur on the Coder loop or the Output Reviewer loop. Direct path: 2 to 5 seconds. Clarify path: 1 to 2 seconds. Latencies are subject to provider-side queueing and are unlikely to improve without provider-level Latency-Optimized inference modes.

18.2 Cost Per Turn

Approximate cost per turn at current Anthropic pricing: 0.001 to 0.002 USD for direct and clarify paths; 0.02 USD for the SQL happy case; up to 0.15 USD on the worst case with full retries. Worst-case cost is bounded because retry budgets are bounded; this is a direct consequence of the rule-based Orchestrator design choice.

Table 14 Per-Turn Cost Decomposition

Path / Scenario	Coder calls	Code-Rev calls	Output-Rev calls	Writer calls	Approx. cost
direct path	0	0	0	1	≈ $0.001
clarify path	0	0	0	0	≈ $0.0005
sql happy case	1	1	1	1	≈ $0.02
sql + 1 inner retry	2	2	1	1	≈ $0.04
sql + 1 outer retry	2	2	2	1	≈ $0.05
sql worst-case	5	10	2	2 + supervisor	≈ $0.15

18.3 Worst-Case Token Budget Equation

Equation 5 worst-case turn tokens

$$T_{worst} \approx N_{coder} \cdot (P_{coder} + 900) + N_{review} \cdot (P_{review} + 400) + N_{writer} \cdot (P_{writer} + 800) + N_{viz} \cdot (P_{viz} + 900)$$

where N_x denotes the number of calls to agent x in the worst case, P_x denotes the prompt-side token count for agent x, and the constant in each term is the agent's max_tokens cap. Substituting the budgets in Table 14 with empirical prompt sizes yields a worst-case ceiling on the order of 30,000 to 40,000 tokens per turn — bounded, and at current Anthropic pricing, sub-dollar.

18.4 Known Limitations

- **Groundedness verifies values, not attribution.** A number that appears in the rows but is attributed to the wrong group can pass the check.

- **Planner is a single point of failure for history.** If the Planner misinterprets a follow-up, no downstream agent can recover.
- **Output Reviewer at Haiku can rubber-stamp.** Subtle semantic mismatches occasionally pass through; a Sonnet upgrade is recommended once cost permits.
- **PII is not guarded.** customer_name and customer_email are queryable; see Section 17.4.
- **Sandbox is dev-grade.** The in-process exec sandbox is appropriate for single-user dev; multi-tenant production requires subprocess isolation.
- **Tests are not in CI.** test_golden_sql.py is CI-ready but no CI workflow file is committed.
- **Golden SQL set is starter-size.** 25 entries; should grow as new failure modes surface.
- **Multimodal Viz Reviewer is occasionally flaky.** Acceptable because exhaustion of its loop drops only the chart, not the answer.
- **No alerting on chat_trace patterns.** Manual queries today; a small Grafana dashboard is a near-term improvement.
- **Sonnet at temp 0 isn't strictly deterministic.** Identical-replay testing is not fully reliable.
- **Prompt-version traceability is git-only.** No prompt-version hash stored in chat_trace; a recommended improvement.
- **Cost increased ~3-5× with the Sonnet upgrade.** Justified by the SQL-quality lift but worth re-checking quarterly.

19. Conclusions and Future Directions

The Merchandising AI demonstrates that a focused multi-agent system, built with deliberate rule-based orchestration and aggressive layered evaluation, can reliably translate natural-language business questions into grounded SQL answers at a bounded cost-per-turn. The architectural commitments including single-hub orchestration, two-tier model selection (Sonnet 4.6 for the SQL bottleneck, Haiku 4.5 elsewhere), chain-of-thought scaffolding triplet (reasoning block + reasoning verification + safe-values injection), hard-block groundedness, and end-to-end observability produce a system whose every answer can be audited, every retry can be bounded, and every regression can be detected before it reaches a user.

The deliberate non-choices of no MCP / tool-use loop, no LangChain, no vector database were made because the specific business problem rewards predictability over flexibility. A different problem (open-ended retrieval over policy documents, autonomous multi-step task

execution against multiple APIs) would invite different non-choices. The architecture's value lies in matching its commitments to its problem.

19.1 Future Directions

- **CI integration:** Wire test_golden_sql.py and verify_evals.py into a GitHub Actions workflow so every prompt or model change runs the regression suite before merge.
- **Prompt-version tracking:** Store a hash of every prompt file in chat_trace.steps_json so a regression-bisect can attribute a failure to a specific prompt change.
- **PII redaction:** Wrap sql_executor with a PII-scrub pass that drops customer_name and customer_email from result rows before they reach the LLMs.
- **Subprocess-isolated viz sandbox:** Replace the in-process exec sandbox with a separate-subprocess executor under timeout and memory caps for production multi-tenancy.
- **Vector store for free-text Q&A:** If the scope widens to include questions over uncategorised free-text policy or product-description documents, an embedding store and a retrieval agent become appropriate additions.
- **Per-turn cost ceiling:** Add a hard token-budget check that aborts a turn if the worst-case ceiling has been exceeded, providing a fail-safe against an unbounded retry caused by future code changes.
- **Live Grafana dashboard:** Wire a dashboard against chat_trace showing supervisor invocation rate, groundedness warning rate, average retry count per path, and the most-expensive failures.
- **Output Reviewer to Sonnet:** Promote the Output Reviewer to Sonnet 4.6 once cost permits, closing the rubber-stamp gap noted in Section 18.4.

20. Appendix A — File Manifest

Complete listing of every source file in the non_purchaser_feedback repository, with module size and primary purpose.

Table 15 File Manifest

File	Lines	Purpose
streamlit_app.py	420	Streamlit UI; chat surface + Recommendations view
agent_backend.py	412	UI ↔ backend seam; deterministic Recommendations report
agents/__init__.py	31	Package marker / public re-exports
agents/llm.py	187	aisuite + native anthropic vision facade
agents/schemas.py	83	Shared dataclasses: PlannerOutput, AgentResponse, StepEvent
agents/planner.py	126	sql / direct / clarify routing + history resolution
agents/coder.py	95	NL question → SQL with <reasoning> + <sql_query>
agents/code_reviewer.py	90	SQL technical + reasoning verification
agents/output_reviewer.py	66	Semantic-fit and viz-decision review
agents/writer.py	141	Markdown answer composition (sql / direct / clarify)
agents/viz_coder.py	59	matplotlib snippet generation

File	Lines	Purpose
agents/viz_code_reviewer.py	44	Static safety review of viz code
agents/viz_reviewer.py	44	Multimodal review of rendered PNG
agents/supervisor.py	75	Recovery decision-making
agents/orchestrator.py	679	Rule-based control plane — single hub
agents/prompts/*.txt	1109 total	Nine prompt files (one per agent)
tools/__init__.py	60	Public surface re-exports
tools/sql_tools.py	208	Schema text, parser, safety guard, executor
tools/viz_tools.py	129	Viz parser, sandboxed runner, base64 helper
tools/excel_tools.py	75	openpyxl-backed .xlsx bytes
tools/groundedness.py	502	Regex-based numeric groundedness verifier
tools/trace_logger.py	205	chat_trace MySQL persistence
data_prep/01_generate_feedback_data_mysql.py	387	Synthesise 1,200 seed rows
data_prep/02_enrich_topics.py	555	One-time topic enrichment via Haiku 4.5
data_prep/03_eval_topic_classifier.py	231	Offline classifier accuracy + confusion matrix
tests/test_planner.py	71	Planner smoke test
tests/test_orchestrator.py	63	End-to-end smoke test

File	Lines	Purpose
tests/test_golden_sql.py	151	26-entry shape-check regression runner
tests/golden_sql_dataset.jsonl	26 entries	Q→SQL pattern assertions
tests/verify_evals.py	353	7-check integration smoke for evals + persistence
tests/eval_prompts.md	—	15 manual prompts at three difficulty tiers
tests/README.md	—	Operational guide for every test
docs/00_system_overview.md	449	Primary system overview
docs/06_topology_graph.{svg,png,jpg}	—	Component call graph
docs/07_workflow_tree.{svg,png,jpg}	—	Temporal workflow tree
docs/08_prompts_reference.md	1423	Catalogue of every prompt
docs/09_agents_and_tools_reference.md	79	Per-component spec table
docs/10_layered_architecture*.md	2378 total	Three layered-architecture variants
requirements.txt	8	Pinned runtime dependencies
.env / .env.example / .gitignore	—	Configuration plumbing

21. References

- *00_system_overview.md* — Primary system overview document inside the repository.
- *10_layered_architecture_A_full.md* — Full 12-layer canonical reference.
- *10_layered_architecture_B_grouped.md* — 8-layer grouped onboarding walkthrough.
- *10_layered_architecture_C_executive.md* — 5-bucket executive view for stakeholders.
- *09_agents_and_tools_reference.md* — Per-component spec table for every agent and tool.

- *08_prompts_reference.md* — Catalogue of every system prompt in the agents directory.
- *06_topology_graph.{svg,png,jpg}* — Component call graph rendered for Figure 1 of this report.
- *07_workflow_tree.{svg,png,jpg}* — Temporal agentic workflow rendered for Figure 2 of this report.
- *Anthropic Claude API documentation* — claude-haiku-4-5-20251001 and claude-sonnet-4-6 model identifiers.
- *aisuite* — Provider-agnostic LLM client library used to abstract the model provider in agents/llm.py.
- *Streamlit* — Web UI framework used to render the chat surface and Recommendations view.
- *MySQL Connector / Python* — Driver used by sql_executor and trace_logger.
- *Matplotlib* — Headless Agg backend used inside the viz sandbox.
- *openpyxl* — Workbook writer behind to_excel for the chat Excel download affordance.